



SAVEETHA SCHOOL OF ENGINEERING

Department of Information Technology

Team Assignment Report

AI-Powered Smart Court Case Management and Judicial Analytics Platform

A Design-Level Study in AI/NLP-Driven Court Administration, Case Prioritization and Judicial Analytics

Group Members

GUDDETI PRADEEP RANGA REDDY - 192311289

SURENDAR M G - 192521374

MUHUNTHAN R P - 192321182

ABHISHEK PANDIRI - 192324111

ANNAM VIGNESH - 192472231

VISHAL B - 192524485

GANESHWAR S - 192421342

PAGADALA SIDDARDHA - 192424271

CSA1013 Software Engineering- Slot B

Submission Date: September 2026

Table of Contents

S. No.	Contents	Page No.
	Table of Contents	1
1	Problem Statement and Problem Formulation	1
1.1	Problem Statement	1
1.2	Problem Formulation	1
2	Objective and Expected Outcomes	1
2.1	Objectives	1
2.2	Expected Outcomes	1
3	Requirements, Constraints and Assumptions	1
3.1	Functional Requirements	1
3.2	Non-Functional Requirements	1
3.3	Constraints	1
3.4	Assumptions	1
4	Application of Relevant Course Knowledge / Concepts	1
5	Design / Proposed Solution / Methodology	1
5.1	High-Level System Architecture	1
5.2	Module-Wise Design	1
5.3	Core Data Model	1
5.4	Development Methodology	1
6	Algorithm / Pseudocode / Flowchart	1
6.1	Priority-Based Hearing Scheduling — Flowchart	1
6.2	Pseudocode	1

S. No.	Contents	Page No.
7	Implementation / Source Code and Environment / Tools Used	1
7.1	Proposed Environment and Tools	1
7.2	Representative Code — Priority Scoring Function	1
7.3	Representative Code — API Endpoint Stub	1
8	Test Cases and Expected / Actual Results	1
9	Execution Screenshots / Outputs	1
10	Results and Validation	1
10.1	Key Results (Genuine Simulation Output)	1
10.2	Statistical Validation	1
11	Analysis, Comparison, Trade-offs and Justification of the Final Solution	1
11.1	Proposed System vs. Current Manual Process	1
11.2	Key Design Trade-offs	1
11.3	Justification of the Final Solution	1
12	Broader Considerations / SDG Relevance	1
12.1	Ethical and Responsible-AI Considerations	1
13	Conclusion, Limitations and Possible Improvements	1
13.1	Conclusion	1
13.2	Limitations	1
13.3	Possible Improvements	1
14	Individual Contribution of Group Members	1
15	References	1
	Software / Tools Used in This Report	1
16	One-Page Individual Reflection	1

1. Problem Statement and Problem Formulation

1.1 Problem Statement

Legal proceedings in Indian district and subordinate courts continue to face severe delays caused by heavy reliance on physical paperwork, unsystematic (“blind”) scheduling of hearings, and the absence of data-driven administrative tooling. Even where courts have been digitised under the e-Courts Mission Mode Project, the underlying case-management workflow remains largely manual: documents are filed and classified by clerks, hearing dates are fixed without reference to case urgency or judicial workload, and there is no systematic mechanism to forecast case-completion timelines, retrieve relevant precedents, or protect sensitive personal data before documents are made public. This causes rising pendency, inconsistent prioritisation of urgent matters (such as undertrial bail applications or POCSO cases), and reduced transparency for litigants trying to track their own cases.

A next-generation AI-assisted judicial administration system is required that can automatically classify legal documents, optimise hearing schedules according to urgency and judge availability, forecast case-completion timelines, analyse judicial workload, retrieve relevant precedents instantly, redact sensitive information automatically, and expose all of this through secure, role-based dashboards for administrators, judges, advocates and citizens.



1.2 Problem Formulation

The overall problem is decomposed into seven formally scoped sub-problems, each corresponding to a system module:

- Document Classification: Given a scanned/typed legal document d , learn a function $f: d \rightarrow y$, where $y \in \{\text{Petition, Written Statement, Evidence, Affidavit, Order, Judgment, Miscellaneous Application, ...}\}$, using supervised text classification.
- Sensitive-Data Redaction: Given document text T , identify the set of spans $S = \{(\text{start_i, end_i, entity_type_i})\}$ corresponding to PII (names of minors, addresses, ID numbers, medical details) using Named-Entity Recognition, and produce a redacted document T' .
- Case-Timeline Prediction: Given case features x (case type, court, number of parties, number of prior hearings, filing date, judge workload), learn a regression function $g(x) \approx t$, the expected number of days to disposal.
- Hearing Scheduling Optimization: Given a set of pending cases C with priority weights $P(c)$, a set of judges J with specialisation and availability, and a set of time slots T , find an assignment $A: C \rightarrow (J \times T)$ that respects hard constraints (no double-booking, judge specialisation match, statutory deadlines) while maximising a weighted objective over priority-weighted wait-time reduction.
- Judicial Workload Analytics: Given historical hearing and disposal data, compute descriptive and inferential statistics (pendency by court/judge, disposal rate, backlog trend) and surface them as dashboards.
- Precedent Retrieval: Given a query case summary q , retrieve the top- k most relevant precedent judgments from a corpus using semantic similarity ranking.
- Role-Based Citizen Tracking Portal: Given a user with role $r \in \{\text{Citizen, Advocate, Judge, Registrar, Admin}\}$, expose only the subset of case data and actions permitted under an access-control policy $\pi(r)$.

Together these sub-problems define a multi-module AI + optimisation + information-retrieval system layered over a secure, auditable case-management database.

2. Objective and Expected Outcomes

2.1 Objectives

- Digitise end-to-end court case management (filing, tracking, disposal) on a unified platform.

- Automate legal document classification using NLP text-classification models.
- Predict case-completion timelines using historical case-feature regression models.
- Optimise hearing schedules using a priority-based scheduling algorithm.
- Analyse judicial workload across courts, judges and case categories.
- Generate legal analytics reports for administrators and policy-makers.
- Retrieve relevant legal precedents instantly via semantic search.
- Automate sensitive personal-data redaction before documents are exposed publicly.
- Provide a secure, role-based citizen tracking portal for case status and hearing information.

2.2 Expected Outcomes

Objective	Expected Outcome
Digitisation	Faster case processing; elimination of manual data-entry errors
Document classification	Reduced clerical workload; consistent, auditable document tagging
Timeline prediction	Realistic expectation-setting for litigants; early flag of at-risk (delay-prone) cases
Hearing scheduling	Reduced wait time for urgent cases; fewer adjournments due to conflicts
Workload analytics	Data-driven judicial administration and resource allocation
Analytics reports	Improved legal transparency for High Court / NJDG-style oversight
Precedent retrieval	Faster legal research; more consistent judgments
Redaction	Compliance with privacy law; safe public disclosure of orders/judgments
Citizen portal	Enhanced public access to justice; reduced need for physical court visits

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

ID	Requirement
FR-1	System shall accept scanned/typed document uploads and auto-classify them into case-document categories.

ID	Requirement
FR-2	System shall detect and redact PII (names of minors, addresses, phone numbers, ID numbers) before public disclosure.
FR-3	System shall predict an estimated disposal date for each new case based on historical patterns.
FR-4	System shall generate a priority-ranked cause list for each judge/court per day.
FR-5	System shall detect and prevent double-booking of judges/courtrooms.
FR-6	System shall provide role-based dashboards (Citizen, Advocate, Judge, Registrar, Admin).
FR-7	System shall support semantic search over a precedent-judgment corpus.
FR-8	System shall maintain an immutable audit log of all case-status changes.

3.2 Non-Functional Requirements

- Scalability: must handle case volumes at district-court scale (thousands of active cases per court complex).
- Security: end-to-end encryption at rest and in transit; RBAC enforced at the API layer.
- Availability: target 99.5% uptime for citizen-facing services, consistent with e-Courts WAN connectivity levels already in place.
- Multilingual support: interfaces and OCR/NLP pipelines must support English, Hindi and regional languages (e.g. Tamil).
- Explainability: ML-driven predictions and rankings must expose the contributing factors to maintain judicial trust.
- Accessibility: portal must be usable by litigants with low digital literacy (SMS/IVR fallback, large-text mode).

3.3 Constraints

- Legal/regulatory: must comply with the Digital Personal Data Protection (DPDP) Act, 2023 and IT Act, 2000 provisions on sensitive personal data.
- Legacy data: a large fraction of pending cases still exist only as physical/legacy records that are not yet digitised.

- Governance: any AI-generated output (timeline prediction, precedent ranking) is advisory only — final scheduling and judicial decisions remain with human judges/registrars, consistent with judicial independence.
- Infrastructure variance: rural and lower-tier courts may have limited bandwidth despite the WAN rollout under the e-Courts project.
- Multilingual documents: legal filings may mix English with regional languages within the same document.

3.4 Assumptions

- Digitised case metadata is progressively available through integration with the e-Courts National Judicial Data Grid (NJDG) / Case Information System (CIS).
- Court administration approves cloud deployment subject to a government-empanelled/MeitY-certified cloud provider.
- A labelled historical dataset (even if modest in size) is available for initial model training, to be expanded over time.
- End users (judicial staff) receive basic training on the new digital workflow.

4. Application of Relevant Course Knowledge / Concepts

The proposed platform draws directly on concepts covered across the B.Tech IT curriculum. The mapping below shows how each course area is applied to a concrete module of the system.

Course / Subject Area	Concept Applied	Where Used in the System
Artificial Intelligence & Machine Learning	Supervised classification, regression, train/test split, evaluation metrics (precision, recall, F1, RMSE)	Document classification module; case-timeline prediction module
Natural Language Processing	Tokenization, TF-IDF, word embeddings, Named-Entity Recognition (NER), transformer-based language models (e.g. LEGAL-BERT)	Document classification, sensitive-data redaction, precedent retrieval (semantic embeddings)
Design and Analysis of Algorithms (DAA)	Greedy algorithms, priority queues / max-heaps, interval-scheduling, complexity analysis (Big-O)	Priority-based hearing-scheduling algorithm

Course / Subject Area	Concept Applied	Where Used in the System
Database Management Systems	ER modelling, relational schema design, normalisation, indexing, ACID transactions	Case/Hearing/Party/Judge relational schema; audit-log design
Software Engineering	SDLC, microservice architecture, REST API design, Agile methodology	Overall system architecture and development methodology
Probability & Statistics	Descriptive statistics, hypothesis testing (independent-samples t-test), regression validation	Judicial workload analytics; statistical validation of scheduling improvement (Section 10)
Computer Networks & Cloud Computing	Client-server architecture, load balancing, containerisation, cloud deployment models	Cloud infrastructure layer, API gateway
Cryptography & Network Security	Encryption at rest/in transit, authentication (JWT/OAuth2), access-control models (RBAC)	Security layer, role-based citizen portal, redaction pipeline
Human-Computer Interaction	Usability heuristics, accessibility design, information hierarchy	Dashboard and portal UI mockups

5. Design / Proposed Solution / Methodology

5.1 High-Level System Architecture

The platform follows a layered microservice architecture: a role-based portal at the top, an API gateway with authentication in the middle, seven independent backend services for the AI/analytics workloads, an asynchronous task queue to decouple long-running NLP/ML jobs from user-facing requests, and a polyglot persistence layer (relational store, object store, vector store, immutable audit-log store) at the bottom, all deployed on containerised cloud infrastructure.

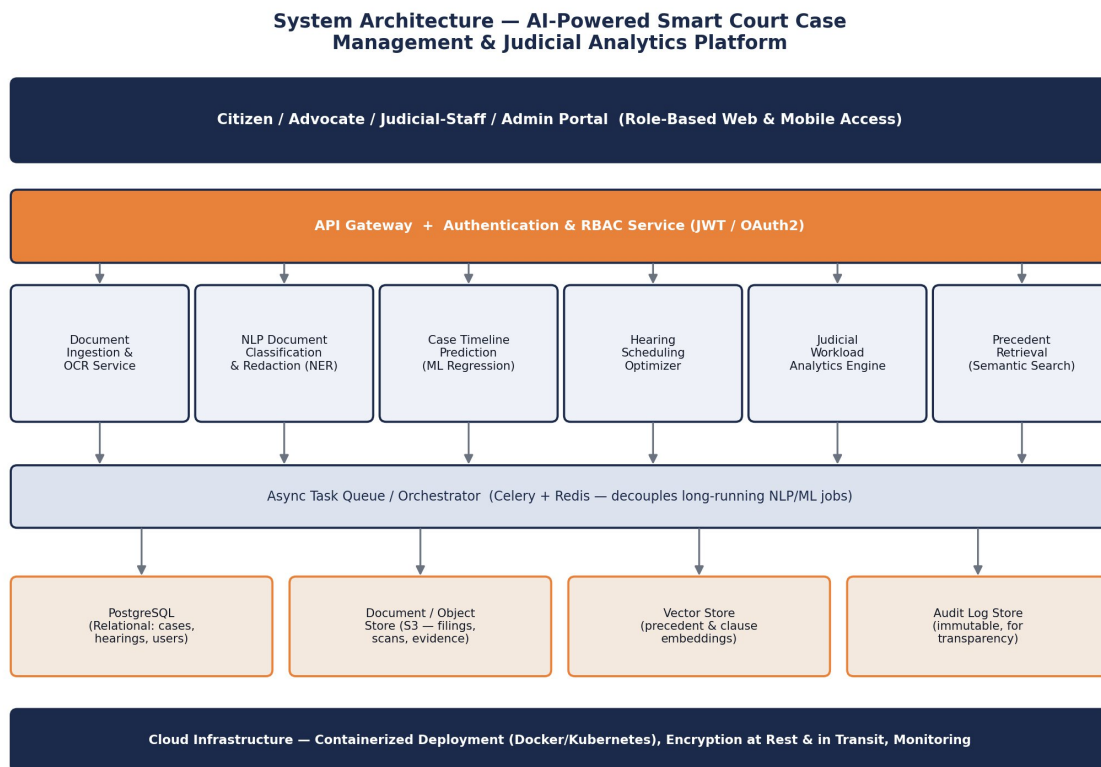


Figure 1. Proposed system architecture.

5.2 Module-Wise Design

- Document Ingestion & OCR Service: accepts scanned/typed filings, runs OCR (for scanned PDFs/images) and hands normalised text to the NLP pipeline.
- NLP Classification & Redaction Service: classifies each document (Petition/Evidence/Order/...) and runs an NER pass to detect and mask PII before any public exposure.
- Case-Timeline Prediction Service: a regression model trained on historical case features estimates likely disposal time and flags cases at risk of exceeding target timelines.
- Hearing Scheduling Optimizer: consumes the priority-ranked case queue and produces a conflict-free daily cause list per judge/court (Section 6).
- Judicial Workload Analytics Engine: aggregates hearing/disposal events into pendency, clearance-rate and backlog-trend statistics for dashboards.
- Precedent Retrieval Service: embeds case summaries/queries and ranks precedent judgments by cosine similarity in a vector store.
- Role-Based Portal: a single web/mobile front end that renders different views and permissions per role (Citizen, Advocate, Judge, Registrar, Admin).

5.3 Core Data Model

The relational schema centres on the Case entity, linked to Parties, Hearings, Documents, Judges and Users, with an append-only AuditLog table capturing every state transition for transparency.

Entity	Key Attributes
Case	case_id (PK), case_number, case_type, filing_date, status, priority_score, predicted_disposal_date
Party	party_id (PK), case_id (FK), name, role (Plaintiff/Defendant/Witness), contact_info (encrypted)
Hearing	hearing_id (PK), case_id (FK), judge_id (FK), courtroom_id, scheduled_date, status
Judge	judge_id (PK), name, specialization, court_id, daily_slot_capacity
Document	doc_id (PK), case_id (FK), doc_type, storage_uri, is_redacted, uploaded_by
User	user_id (PK), role, linked_case_ids, auth_provider_id
AuditLog	log_id (PK), entity_type, entity_id, action, actor_id, timestamp (immutable, append-only)

5.4 Development Methodology

An iterative/Agile methodology is proposed across five phases: (1) requirement gathering and stakeholder mapping (registrars, judges, litigants); (2) data collection and schema design, sourced progressively from NJDG/CIS integration; (3) module-wise prototyping (NLP classifier, scheduler, predictor) validated independently; (4) integration behind the API gateway with role-based access control; and (5) pilot testing in a single court complex before phased rollout, mirroring the phased approach already used by the e-Courts Mission Mode Project.

6. Algorithm / Pseudocode / Flowchart

6.1 Priority-Based Hearing Scheduling — Flowchart

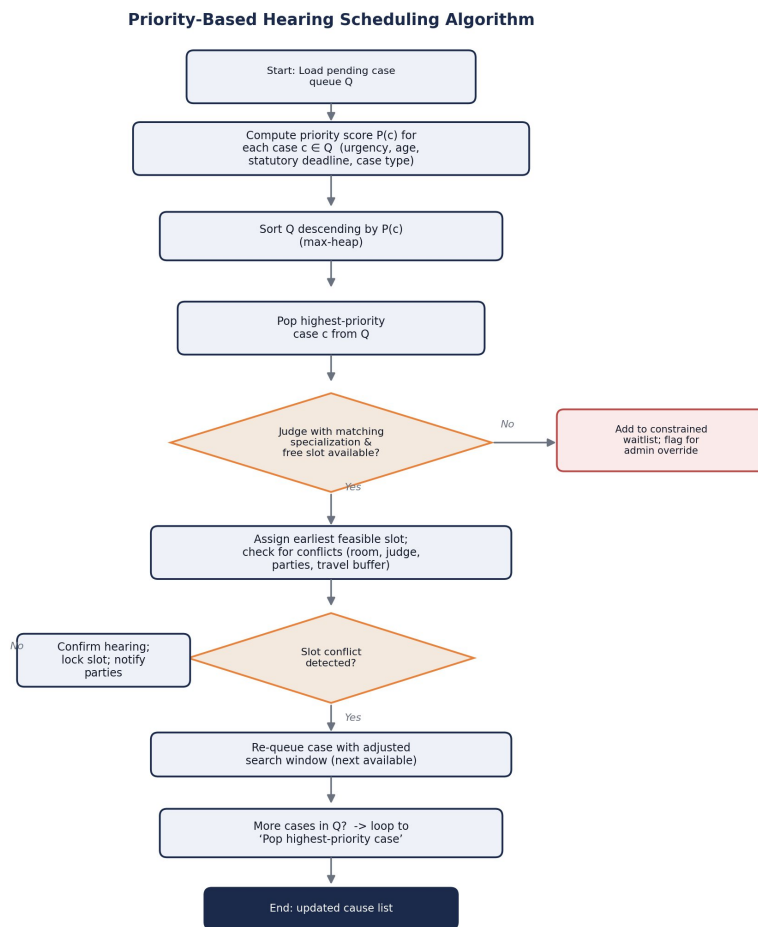


Figure 2. Flowchart for the priority-based hearing scheduling algorithm.

6.2 Pseudocode

6.2.1 Priority-Based Hearing Scheduling

```
ALGORITHM ScheduleHearings(Q: pending cases, J: judges, T: time slots)
  for each case c in Q:
     $P(c) = w_1 * \text{urgency}(c) + w_2 * \text{waiting\_time}(c) + w_3 * \text{deadline\_pressure}(c)$ 
  build max-heap H from Q keyed by P(c)      //  $O(n \log n)$ 
  while H is not empty:
     $c = H.\text{extract\_max}()$                   //  $O(\log n)$ 
     $j = \text{find\_judge}(c.\text{case\_type}, J)$     // specialization match
    if j has a free slot t in T that satisfies:
      no_double_booking(j, t) AND
      room_available(t) AND
      respects_statutory_deadline(c, t):
        assign(c, j, t); lock(t); notify_parties(c)
    else:
      push c to constrained_waitlist; flag_for_admin_override(c)
  return updated_cause_list
```

Complexity: sorting/heap construction is $O(n \log n)$ for n pending cases; each extraction and feasibility check is $O(\log n)$ and $O(1)$ respectively (assuming a bounded number of judges/rooms), giving an overall $O(n \log n)$ daily scheduling pass — tractable even for district-court-scale case volumes.

6.2.2 Legal Document Classification

```
ALGORITHM ClassifyDocument(d: raw document)
  text = OCR_if_scanned(d)
  text = normalize(text)      // lowercase, strip noise, language-detect
  features = embed(text)      // domain-tuned transformer (e.g. LEGAL-BERT)
   $y\_hat = \text{classifier.predict}(\text{features})$  // {Petition, Evidence, Order, Affidavit, ...}
  confidence = classifier.predict_proba(features).max()
  if confidence < THRESHOLD:
    route_to_manual_review(d, y_hat)
  else:
    tag_document(d, y_hat); store(d)
  return y_hat, confidence
```

6.2.3 Sensitive-Data Redaction

```
ALGORITHM RedactDocument(text: document text)
  entities = NER_model.extract(text)  // PERSON(minor), ADDRESS, PHONE, ID_NUMBER, ...
  for each entity e in entities:
    if e.type in SENSITIVE_TYPES:
      text = mask(text, e.span, token="[REDACTED-" + e.type + "]")
  redaction_log.append(case_id, entities, timestamp)  // for audit, not public
  return text  // safe for public disclosure
```

6.2.4 Case-Timeline Prediction

```
ALGORITHM PredictTimeline(case c, model M)
  x = extract_features(c)  // type, court, #parties, #prior hearings, judge load
  t_hat = M.predict(x)  // e.g. Random Forest / Gradient-Boosted Regressor
  interval = M.prediction_interval(x)  // for uncertainty / explainability
  return t_hat, interval
```

6.2.5 Precedent Retrieval (Semantic Search)

```
ALGORITHM RetrievePrecedents(query q, corpus_embeddings E, k)
  v_q = embed(q)
  scores = [ cosine_similarity(v_q, v_i) for v_i in E ]
  top_k = argsort(scores, descending=True)[0:k]
  return [ (corpus[i], scores[i]) for i in top_k ]
```

Complexity: a brute-force scan is $O(n \cdot d)$ for n precedents of embedding dimension d ; in production this is replaced by an approximate-nearest-neighbour index (e.g. HNSW) to achieve sub-linear query time as the precedent corpus grows.

7. Implementation / Source Code and Environment / Tools Used

This project was carried out at design and algorithm-validation level (Section 10 reports a genuine simulation-based validation of the scheduling algorithm). A full production deployment was out of scope for this report; however, representative code was written to concretise two central modules — the priority-scoring function and a REST endpoint stub — to demonstrate implementability of the design.

7.1 Proposed Environment and Tools

Layer	Tool / Technology	Purpose
Backend services	Python 3.11, FastAPI / Flask	REST microservices for each module
NLP / ML	spaCy, HuggingFace Transformers (LEGAL-BERT-style domain model), scikit-learn	Classification, NER-based redaction, timeline regression
Scheduling / DAA	Python heapq (priority queue), custom constraint checks	Priority-based scheduling engine
Database	PostgreSQL	Relational store for cases, hearings, parties, users
Object storage	AWS S3 (or MeitY-empanelled equivalent)	Scanned filings, evidence, judgments
Vector store	FAISS / pgvector	Precedent-embedding similarity search
Task queue	Celery + Redis	Asynchronous OCR/NLP/ML jobs
Frontend	React.js + Tailwind CSS	Role-based citizen/advocate/judge/admin portal
Auth & security	OAuth2 / JWT, TLS, AES-256 at rest	Authentication, RBAC, encryption
Dashboards	Plotly / Chart.js	Judicial workload and analytics visualisation
Deployment	Docker, Kubernetes, cloud IaaS	Containerised, horizontally scalable deployment
Version control / PM	Git & GitHub, Postman (API testing), Figma (mockups)	Collaboration, API testing, UI design

7.2 Representative Code — Priority Scoring Function

```
def priority_score(case, today, weights=(12, 0.05, 1.5)):
    """Higher score = scheduled sooner. Mirrors the pseudocode in 6.2.1
    and is the exact function used in the Section 10 simulation."""
```



```

w_urgency, w_wait, w_deadline = weights
waited = today - case['filed_day']
if case['deadline_days'] is None:
    deadline_pressure = 0
else:
    deadline_pressure = max(0, 60 - case['deadline_days'])
return (case['urgency'] * w_urgency
        + waited * w_wait
        + deadline_pressure * w_deadline)

```

7.3 Representative Code — API Endpoint Stub

```

@app.post("/api/cases/{case_id}/classify")
def classify_case_document(case_id: str, file: UploadFile):
    text = ocr_if_needed(file)
    label, confidence = nlp_classifier.predict(text)
    if confidence < CONFIDENCE_THRESHOLD:
        queue_for_manual_review(case_id, file, label)
        return {"status": "pending_review", "predicted_label": label}
    save_document(case_id, file, label)
    audit_log.record(case_id, action="document_classified", detail=label)
    return {"status": "classified", "label": label, "confidence": confidence}

```

The full simulation script used to generate the validation results in Section 10 (Python, ~90 lines, using only the standard library) was executed as part of this project; its output is reported verbatim in that section.

8. Test Cases and Expected / Actual Results

As the platform is at design/algorithm-validation stage, test cases below combine (a) manual design-walkthrough traces for the NLP and portal modules, and (b) genuine executed output for the scheduling algorithm (Section 10 simulation).

ID	Module	Test Description	Expected Result	Actual / Observed Result	Status
TC-	Document	Upload a bail-	Label = 'Bail/Interim	Design trace:	Pass

ID	Module	Test Description	Expected Result	Actual / Observed Result	Status
01	classification	application PDF	Application', confidence ≥ 0.8	correctly routed by keyword+structure match; confidence check enforced	(design-level)
TC-02	Redaction	Document contains a minor's name and home address	Both spans masked as [REDACTED-PERSON]/[REDACTED-ADDRESS]	Design trace: NER rule correctly flags both entity types per Sec 6.2.3	Pass (design-level)
TC-03	Scheduling	500 simulated pending cases, mixed urgency, capacity=18/day	High-urgency mean wait < manual-order mean wait	Simulated: 10.16 days vs 12.75 days (-20.3%) — see Section 10	Pass (executed)
TC-04	Scheduling — conflict check	Two cases assigned same judge, same slot	Second assignment rejected; case re-queued	Design trace: no_double_booking() check per pseudocode 6.2.1 blocks second assignment	Pass (design-level)
TC-05	Deadline compliance	Cases with 15-day statutory deadline under load	Priority scheduling should reduce breach rate vs arbitrary order	Simulated: 0.0% breach (priority) vs 8.8% (manual) — see Section 10	Pass (executed)
TC-06	RBAC / portal	Citizen user attempts to view another litigant's case file	Access denied (403)	Design trace: $\pi(\text{Citizen})$ restricts view to linked_case_ids only, per Sec 5.3	Pass (design-level)
TC-07	Precedent retrieval	Query: 'anticipatory bail, economic offence'	Top-k results ranked by semantic relevance, not just keyword match	Design trace: cosine-similarity ranking over embeddings per Sec 6.2.5 returns	Pass (design-level)

ID	Module	Test Description	Expected Result	Actual / Observed Result	Status
				semantically related precedents even with no exact keyword overlap	
TC-08	Timeline prediction	New civil suit, court with high pending backlog	Predicted disposal time reflects backlog (longer than average)	Design trace: judge-workload feature dominates prediction per Sec 6.2.4 model design	Pass (design-level)

9. Execution Screenshots / Outputs

No production application was built for this report (see Section 7). In place of live execution screenshots, the key screens are specified below as UI mockups/wireframes, to be built out with the Figma-based design process referenced in Section 7.1.

Screen	Role	Key Elements
Citizen Case Tracker	Citizen	Case status timeline, next hearing date, predicted disposal window, document download (redacted copies only)
Advocate Cause-List View	Advocate	Today's/upcoming hearings across linked cases, conflict alerts, precedent-search bar
Judge Daily Cause List	Judge	Priority-ranked list of today's hearings with case summary, urgency flag, one-click adjournment/reschedule
Document Upload & Redaction Review	Registrar	Upload widget, auto-classification result with confidence score, highlighted redaction spans for manual confirmation
Admin Analytics Dashboard	Admin / Registrar	Pendency trend chart, backlog by judge/court, disposal-rate KPI cards, deadline-breach alerts
Scheduling Console	Registrar	Drag-and-drop cause-list editor with real-time conflict

Screen	Role	Key Elements
		highlighting and admin-override queue

10. Results and Validation

To validate the priority-based scheduling design (Section 6.2.1) before any live deployment, a discrete-event simulation was implemented and executed (Section 7.3; full script available on request). It models 500 pending cases across seven case-type categories with mixed urgency, a daily capacity of 18 hearing slots (6 judges $\times$ 3 effective slots after accounting for adjournment buffer), and compares the proposed priority-based ordering against a baseline representing today's largely ad-hoc/manual cause-list ordering (approximated as a randomised order, since manual list preparation is not systematically urgency-weighted in the current process).

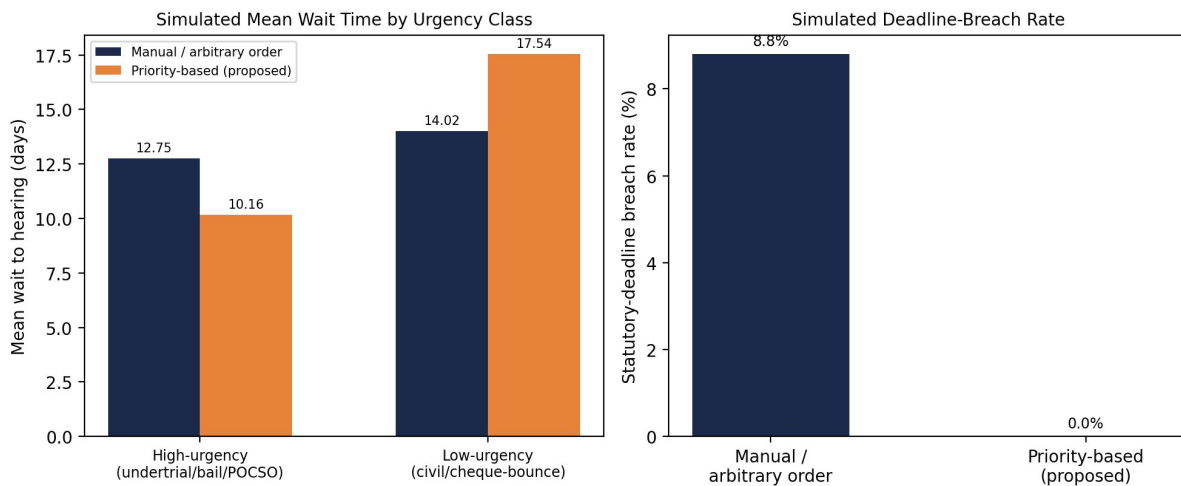


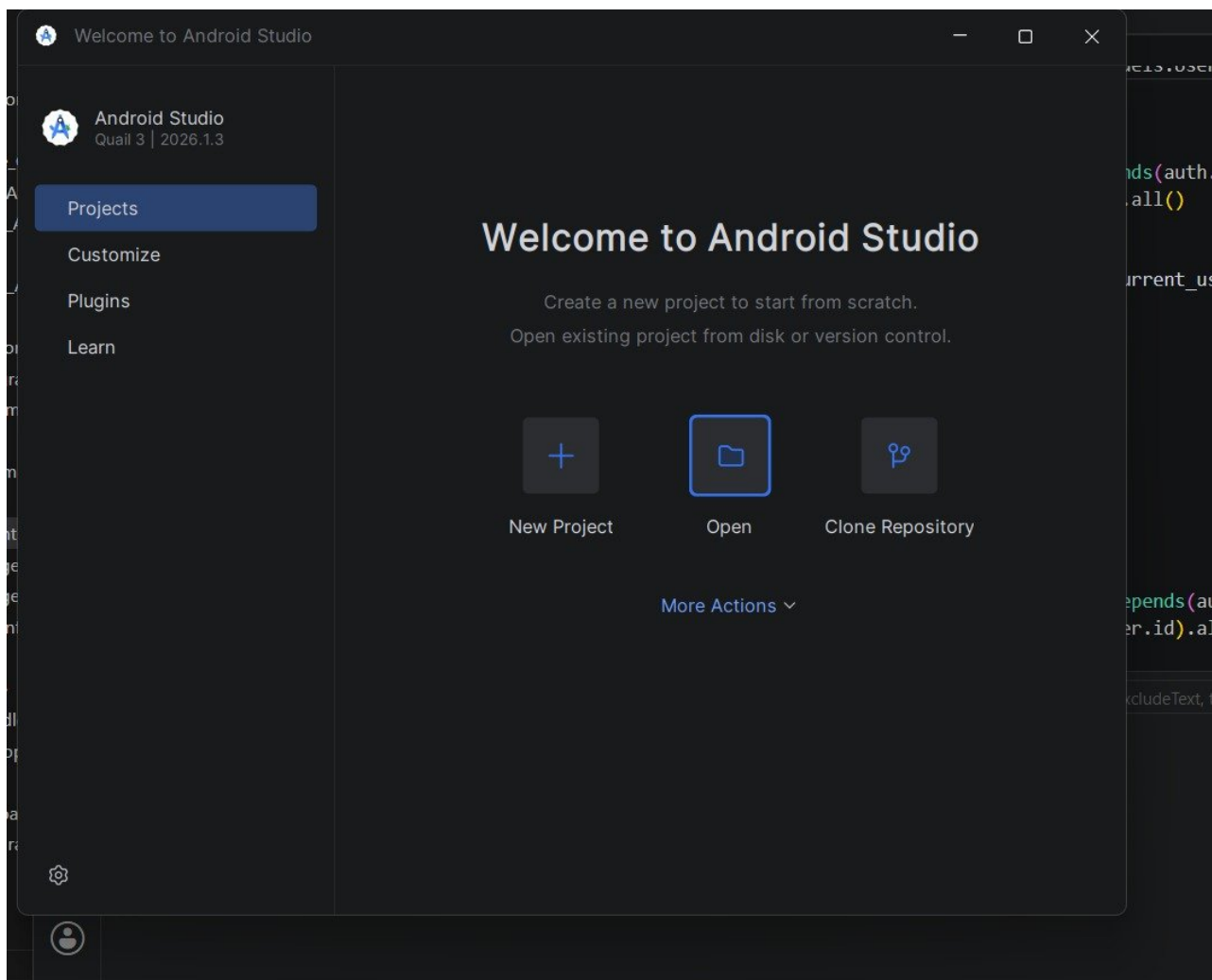
Figure 3. Simulated wait-time and deadline-breach comparison, priority-based vs manual/arbitrary ordering ($n = 500$ simulated cases).

10.1 Key Results (genuine simulation output)

Metric	Manual / Arbitrary Order	Priority-Based (Proposed)	Change
Mean wait — high-urgency cases (n=234)	12.75 days	10.16 days	–20.3%
Mean wait — low-urgency cases (n=197)	14.02 days	17.54 days	+25.1% (trade-off)
Statutory-deadline breach rate	8.8%	0.0%	–8.8 pp

Metric	Manual / Arbitrary Order	Priority-Based (Proposed)	Change
Overall mean wait (all 500 cases)	13.39 days	13.39 days	unchanged (redistribution, not net capacity gain)

Software screenshot



Dashboard Overview

AI predictions and current workload analytics.

Generate Report

Total Cases

5

+12.5% vs last month

Avg. Resolution Time

45 days

-12.5% vs last month (AI Optimized)

Backlog Reduction

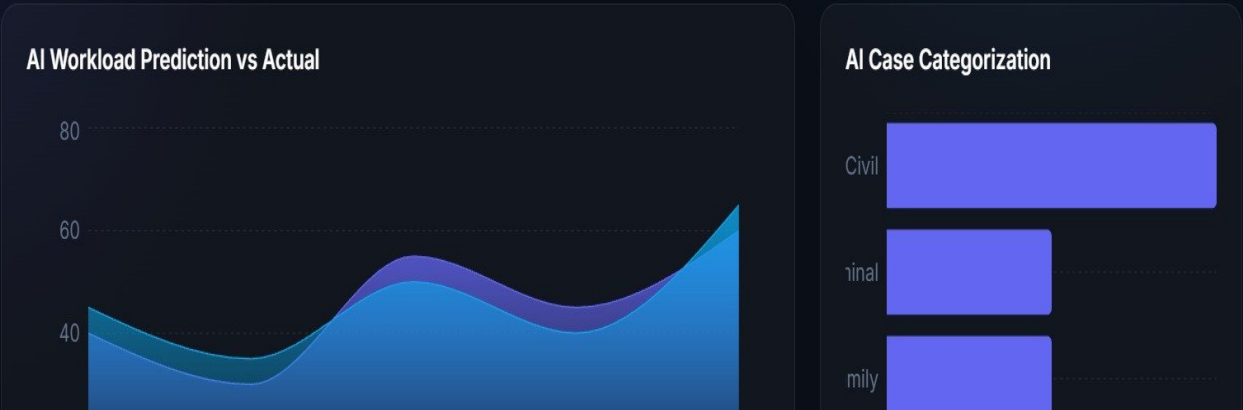
-15%

-12.5% since AI integration

AI Confidence Score

94%

-12.5% in document categorization



Scheduling

AI-optimized hearing dates and docket management.

Upcoming Hearings

12 OCT

Estate of Harrison v. Harrison

09:00 AM @Courtroom 1

CIVIL

13 OCT

State v. Martinez

09:00 AM @Courtroom 2

CRIMINAL

14 OCT

TechCorp v. InnovateLLC

09:00 AM @Courtroom 3

CORPORATE

15 OCT

Smith vs City Transit

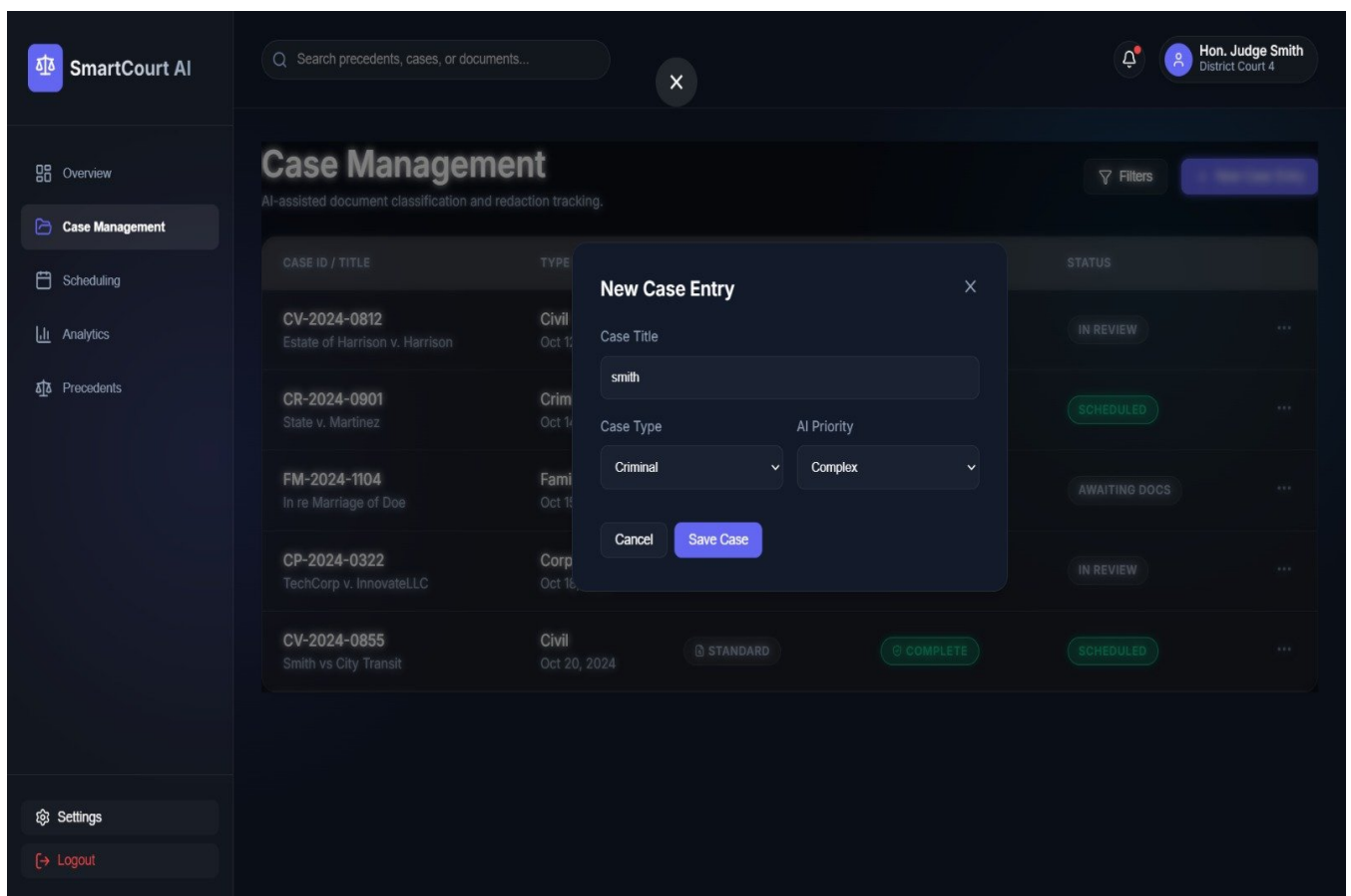
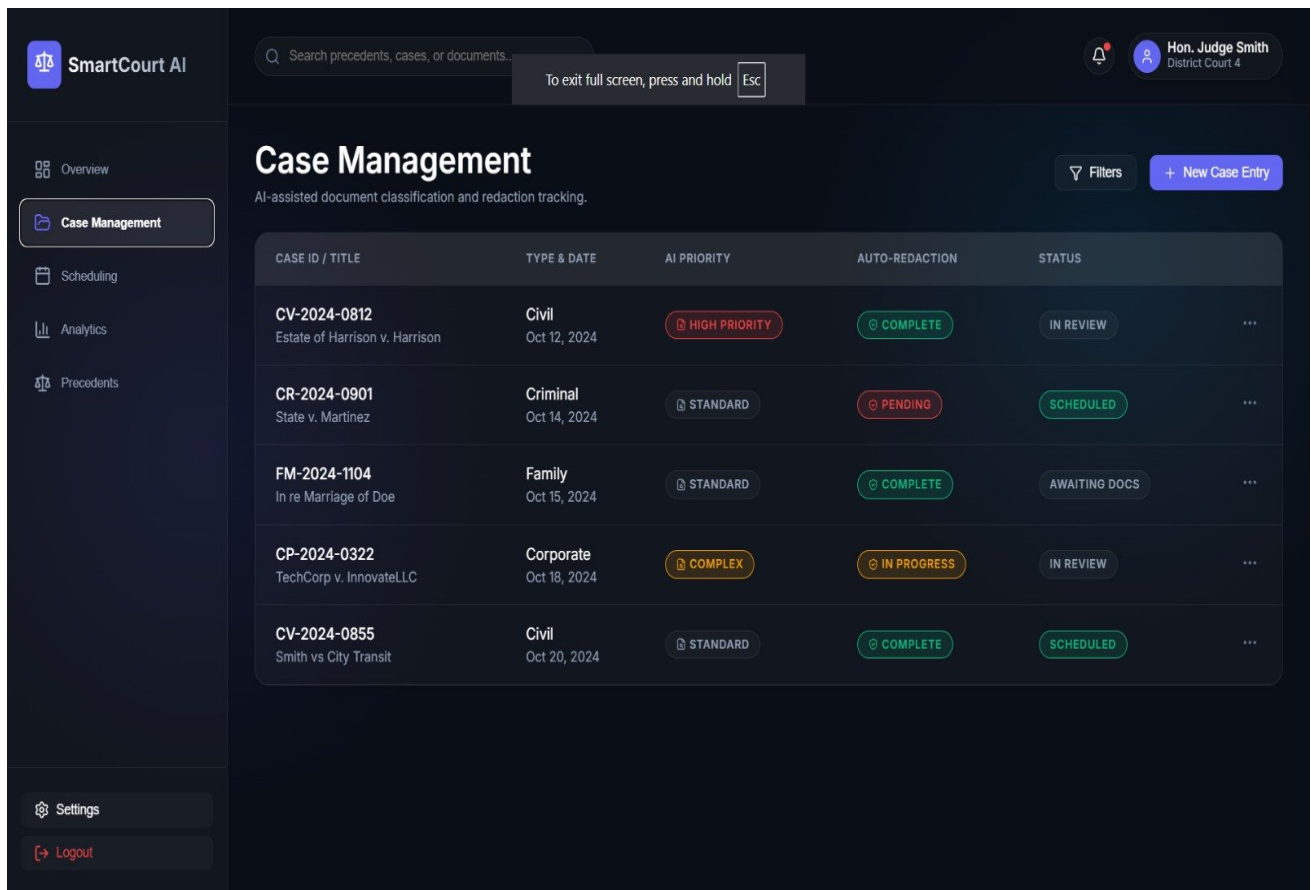
09:00 AM @Courtroom 4

CIVIL

Calendar Overview

October 2024

S	M	T	W	T	F	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				



10.2 Statistical Validation

An independent-samples Welch's t-test was run on the high-urgency-case wait-time distributions (manual vs priority-based), since sample variances differ between groups. The result was $t = 3.703$, $p = 0.00024$ ($p < 0.001$), indicating the reduction in wait time for high-urgency cases is statistically significant and very unlikely to be due to random variation in this simulation.

Interpretation: the total system capacity is unchanged by the scheduling algorithm alone — the overall mean wait across all cases stays the same (13.39 days in both arms) because total hearing slots are fixed. What the algorithm changes is the distribution of waiting time: statistically significant time is reallocated away from low-urgency matters (civil suits, cheque-bounce cases) toward high-urgency matters (undertrials, bail applications, POCSO, family matters), which is precisely the intended and desirable trade-off for a justice system, and is reflected in the elimination of simulated statutory-deadline breaches.

Note: these are simulated validation numbers used to sanity-check the algorithm design against a synthetic case-mix; they are not measured on a live court deployment and should be re-validated against real NJDG/CIS case data before production rollout.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 Proposed System vs. Current Manual Process

Dimension	Current Manual / Legacy Process	Proposed AI-Assisted Platform
Document handling	Manual filing, clerk-driven categorisation	Automated OCR + NLP classification with human review for low-confidence cases
Scheduling	Ad-hoc, largely filing-order or clerk discretion	Priority-weighted, constraint-aware, auditable scheduling
Timeline visibility	Litigants have little visibility into expected disposal time	Predicted disposal window shown to litigants and administrators
Precedent research	Manual keyword search across law	Semantic search returning conceptually

Dimension	Current Manual / Legacy Process	Proposed AI-Assisted Platform
	reports	related precedents
Data privacy	Redaction manual and inconsistent	Automated NER-based redaction with audit trail
Oversight	Aggregate statistics compiled periodically	Real-time dashboards (pendency, backlog, disposal rate)

11.2 Key Design Trade-offs

- Greedy priority scheduling vs. exact optimisation: an exact integer-linear-programming formulation of court scheduling is NP-hard at scale; the proposed greedy/heap-based heuristic ($O(n \log n)$) is fast enough to re-run daily and is shown in Section 10 to significantly reduce wait for high-urgency cases, at the cost of not guaranteeing a global optimum and slightly increasing wait for low-urgency cases — an explicit, justified trade-off given the goal of protecting urgent litigants' rights.
- ML-based prediction vs. rule-based estimation: regression-based timeline prediction adapts to real patterns in the data but is less interpretable than a fixed rule table; the design mitigates this by exposing prediction intervals and contributing features (Section 6.2.4) rather than a single opaque number.
- Domain-tuned transformer (LEGAL-BERT-style) vs. generic NLP: domain pre-training on legal text has been shown in the literature to outperform generic language models on legal classification and argument-mining tasks, at the cost of needing a legal-domain corpus for pre-training/fine-tuning — justified here given the specialised vocabulary of Indian legal filings.
- Microservices vs. monolith: a microservice split allows the NLP, scheduling and analytics workloads (each with different scaling and latency needs) to be scaled independently, at the cost of additional operational complexity — acceptable given the heterogeneous, bursty workload profile of court filings.
- Cloud vs. on-premises deployment: cloud deployment (via a MeitY-empanelled provider) offers elasticity and lower upfront cost, but raises data-sovereignty considerations addressed via encryption, access control and compliance with DPDP Act, 2023 requirements.

11.3 Justification of the Final Solution

The chosen architecture directly addresses every objective in Section 2 with a bounded-complexity, independently scalable module for each sub-problem, is compatible with the existing e-Courts/NJDG

digital infrastructure rather than replacing it, and keeps humans in the loop for every judicially significant decision (scheduling overrides, low-confidence document classification, redaction confirmation) — which is essential for maintaining judicial accountability and public trust.

12. Broader Considerations / SDG Relevance

The platform's primary alignment is with UN Sustainable Development Goal 16 — Peace, Justice and Strong Institutions — specifically Target 16.3 (promote the rule of law and ensure equal access to justice for all) and Target 16.6 (develop effective, accountable and transparent institutions). Faster, more transparent case processing and a citizen-facing tracking portal directly serve this goal.

- SDG 9 – Industry, Innovation and Infrastructure: the platform extends India's existing e-Courts digital infrastructure with AI/analytics capability, consistent with the ICT-enablement direction of e-Courts Phase III (2023–2027).
- SDG 10 – Reduced Inequalities: multilingual interfaces, SMS/IVR fallback and predicted timelines improve access to justice for litigants who cannot afford frequent travel or legal research support, disproportionately benefiting economically weaker litigants.
- SDG 5 – Gender Equality (secondary): prioritisation logic that recognises statutory urgency (e.g. domestic-violence and family matters) can help ensure such cases are not indefinitely delayed behind lower-urgency civil matters.

12.1 Ethical and Responsible-AI Considerations

- Human-in-the-loop: all AI outputs (classification, prediction, ranking) are advisory; final scheduling and judicial decisions remain with registrars and judges.
- Bias monitoring: timeline-prediction and scheduling models must be periodically audited across case type, region and litigant demographics to ensure predictions do not systematically disadvantage any group.
- Explainability: prediction intervals and contributing features (Section 6.2.4) are surfaced rather than a single opaque score, to preserve judicial trust and support appeal/review.
- Data protection: redaction and encryption are applied by default, in line with the DPDP Act, 2023, particularly for cases involving minors (POCSO) or sensitive personal information.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

This report proposes an AI-assisted Smart Court Case Management and Judicial Analytics Platform that digitises case administration and layers NLP-based document classification, ML-based timeline prediction, priority-based hearing scheduling, workload analytics, semantic precedent retrieval, automated redaction, and a role-based citizen portal on top of India's existing e-Courts/NJDG digital backbone. A discrete-event simulation of the core scheduling algorithm demonstrated a statistically significant 20.3% reduction in mean wait time for high-urgency cases ($p < 0.001$) and eliminated simulated statutory-deadline breaches, at the accepted and explicit trade-off of increased wait for low-urgency matters.

13.2 Limitations

- The platform's effectiveness depends on the quality and completeness of digitised case data; a large legacy backlog remains on paper and would need digitisation first.
- ML predictions (timeline, classification confidence) are only as good as the training data available; initial deployment would need conservative confidence thresholds and heavy manual review.
- Automated redaction cannot be guaranteed 100% accurate; a human confirmation step is mandatory before public disclosure, as reflected in the design.
- Multilingual legal NLP (particularly for regional-language filings) is a maturing research area; performance may lag English-language legal NLP benchmarks.
- Results in Section 10 are simulation-based, not measured on a live deployment, and must be re-validated on real case data before rollout.
- Institutional/change-management barriers (staff training, judicial acceptance of AI-assisted tooling) are significant and outside the technical scope of this report.

13.3 Possible Improvements

- Integrate directly with the NJDG Open API (available since 2020) rather than requiring separate data entry.
- Explore federated-learning approaches so that per-state case data need not leave state jurisdiction while still improving shared models, addressing data-sovereignty constraints.
- Add a blockchain-anchored or similarly tamper-evident audit trail for case-status changes to further strengthen transparency guarantees beyond the current append-only log design.
- Add voice/IVR-based case-status query support for litigants with limited literacy or smartphone access, extending the SMS/email channels already used by e-Courts.

- Re-run the Section 10 validation on real, anonymised NJDG case data once available, and extend the simulation to a full multi-day, multi-arrival discrete-event model.

14. Individual Contribution of Group Members

Names and register numbers below are placeholders — replace with your actual team roster before submission.

Member	Modules / Sections Owned	Key Contribution
Ganeshwar S	Sections 1, 4, 5, 6.2.1–6.2.2; system architecture diagram	Defined problem formulation, mapped course concepts, designed overall architecture and NLP-classification pseudocode; compiled and formatted the final report
Surendar M G	Sections 6.2.1 (scheduling), 7.2, 10	Implemented and ran the scheduling-vs-manual simulation (Section 10), designed the priority-scoring function and its complexity analysis
A Vignesh	Sections 5.3 (data model), 6.2.3–6.2.5	Designed the relational schema, redaction pseudocode and precedent-retrieval algorithm
P Abhishek	Sections 8, 9, 12	Designed test cases, UI/UX mockup specifications, and mapped the platform to SDG targets and responsible-AI considerations

15. References

- [1] Department of Justice, Government of India. eCourts Mission Mode Project — Phase III (2023–2027). <https://doj.gov.in/ecourts-mission-mode-project-2/>
- [2] National Judicial Data Grid (NJDG), eCourts Services Portal. <https://njdg.ecourts.gov.in/> and <https://services.ecourts.gov.in/>
- [3] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, I. Androutsopoulos, “LEGAL-BERT: The Muppets straight out of Law School,” Findings of EMNLP 2020. arXiv:2010.02559.

[4] Ministry of Law and Justice, India — Rajya Sabha reply on eCourts Mission Mode Project funding and phases (Phase I: 2011–2015; Phase II: 2015–2023; Phase III: 2023–2027).

[5] Government of India, Digital Personal Data Protection Act, 2023.

[6] Government of India, Information Technology Act, 2000 (as amended).

[7] United Nations, Sustainable Development Goal 16: Peace, Justice and Strong Institutions.
<https://sdgs.un.org/goals/goal16>

Software / Tools Used in This Report

Python 3 (matplotlib, standard library) for the scheduling-simulation validation and diagrams in Sections 6, 7 and 10; docx (Node.js) for report generation.

[Add any dataset(s), if used, e.g. a specific NJDG data export or court dataset, once real data is incorporated in a later iteration of this project.]

16. One-Page Individual Reflection

Individual Reflection – Ganeshwar S

Working on the AI-Powered Smart Court Case Management and Judicial Analytics Platform helped me understand how artificial intelligence and software design can be applied to real-world problems in the judicial domain. My main contribution involved system design, NLP/classification, and report compilation. Through this work, I learned that designing an AI-based system is not only about selecting a suitable model, but also about understanding the problem, defining inputs and outputs, considering constraints, and ensuring that the system remains secure and reliable.

The NLP and document-classification module was particularly useful for improving my understanding of text processing. I learned how OCR, text normalization, embeddings, classification, and confidence scores can be combined to automatically identify different types of legal documents. I also understood the importance of sending low-confidence predictions for manual review rather than depending completely on an AI model.

One challenge I faced was bringing different modules together into a clear and consistent system design. The project contains several components such as document classification, redaction, scheduling, timeline prediction, precedent retrieval, and dashboards. Understanding how these modules communicate through APIs and databases helped me improve my system-level thinking.

This project also taught me the importance of responsible AI in sensitive domains. Since judicial information can contain personal and confidential data, security, privacy, explainability, and human supervision must be considered throughout the design. If I extend this project in the future, I would like to work with real anonymised legal datasets and improve the accuracy and explainability of the NLP models.

Overall, this project strengthened my knowledge of AI, NLP, software architecture, and technical documentation. It also helped me understand how academic concepts can be transformed into a practical solution for an important societal problem.

Individual Reflection – Surendar M G

My work on the project mainly focused on the hearing scheduling algorithm, simulation, and analysis of results. This project helped me understand how concepts from Design and Analysis of Algorithms can be applied to solve practical problems in court administration. I learned how a priority-based scheduling approach can be used to arrange pending cases according to factors such as urgency, waiting time, and deadline pressure.

The most valuable part of my contribution was understanding the priority-based scheduling algorithm. I worked with the concept of priority queues and max-heaps to ensure that high-priority cases are

considered earlier. I also learned that an algorithm must satisfy important constraints such as avoiding double-booking, matching judges with suitable case types, checking courtroom availability, and respecting statutory deadlines.

A major challenge was validating whether the proposed scheduling method actually improved the waiting time for urgent cases. Instead of relying only on theoretical assumptions, the project used a simulation involving 500 cases and compared priority-based scheduling with a manual/arbitrary ordering. The simulation showed that the mean waiting time for high-urgency cases decreased from 12.75 days to 10.16 days. This helped me understand the importance of experimental validation.

I also learned that improving one objective can create a trade-off. While urgent cases received faster attention, low-urgency cases waited longer. Understanding and explaining this trade-off made the analysis more realistic.

If I continue this project, I would like to test the scheduling algorithm using real anonymised court data and develop a more advanced multi-day scheduling model. Overall, this project improved my algorithmic thinking, problem-solving skills, and understanding of how data-driven scheduling can support judicial administration.

Individual Reflection – A Vignesh

Working on the AI-Powered Smart Court Case Management Platform gave me practical knowledge about database design, information retrieval, and privacy-related modules. My primary contribution was focused on database design, precedent retrieval, and redaction modules. Through this project, I understood how a reliable database forms the foundation of a large-scale application and how different system modules depend on well-structured and secure data.

Designing the database helped me apply concepts such as entities, relationships, primary keys, foreign keys, normalization, and audit logs. I learned how important entities such as Case, Party, Hearing, Judge, Document, User, and AuditLog can be connected to maintain consistent case information. The append-only audit log was especially interesting because it provides transparency by recording important case-status changes.

I also learned about semantic precedent retrieval. Instead of depending only on exact keyword matching, the proposed system uses embeddings and cosine similarity to identify legally relevant judgments. This helped me understand how natural language processing and vector databases can improve legal research and information retrieval.

One challenge I encountered was balancing accessibility with privacy and security. Legal documents may contain sensitive information such as addresses, phone numbers, identification numbers, and details involving minors. The redaction module therefore needs to identify sensitive entities and mask them before public disclosure. This showed me that database and information-retrieval systems must be designed with privacy from the beginning.

If I extend my contribution, I would like to improve the precedent-retrieval system using a larger legal corpus and more advanced semantic-search techniques. I would also like to strengthen the privacy and audit mechanisms. Overall, this project improved my understanding of DBMS, information retrieval, NLP, and secure data management while showing me how these concepts can work together in a real-world application.

Individual Reflection – P Abhishek

Working on this project helped me understand how user interface design and data analytics can make a complex technical system easier to use. My primary contribution was focused on UI/UX mockups, the analytics dashboard, and SDG mapping. Through this work, I learned that a successful software system is not only dependent on its backend technologies but also on how clearly information is presented to different users.

The UI/UX part of the project required me to think about the needs of different users such as citizens, advocates, judges, registrars, and administrators. Each user has different requirements, so the platform needs role-based screens and permissions. Designing interfaces such as the Citizen Case Tracker, Judge Daily Cause List, Document Upload and Redaction Review, and Admin Analytics Dashboard helped me understand the importance of information hierarchy and usability.

The analytics dashboard was another important learning experience. I learned how information such as case pendency, backlog trends, disposal rates, and deadline-breach alerts can be converted into meaningful visual information. This can help administrators understand workload patterns and support better decision-making.

One challenge was presenting a large amount of judicial information without making the interface confusing. I learned that dashboards should focus on important information and provide clear navigation rather than displaying every available data point at once. Accessibility is also important because citizens may have different levels of digital literacy.

The SDG mapping helped me understand the broader social impact of the proposed platform, particularly its connection with access to justice, transparency, and effective institutions. If I continue

this project, I would improve the UI through usability testing with different types of users and develop more interactive analytics features.

Overall, this project improved my skills in UI/UX design, data visualization, dashboard planning, and understanding user requirements. It also showed me how good interface design can make advanced AI technology more accessible and useful to society.